

Princípios de Design

Introdução

Os Princípios de Design são fundamentais para a construção de sistemas de software eficientes, flexíveis e fáceis de manter. Nesta aula, vamos explorar quatro conceitos-chave: acoplamento, coesão, decomposição e generalização. Cada um desses princípios ajuda a definir como as classes e os módulos de um sistema interagem e como suas responsabilidades são organizadas. Compreender esses elementos permitirá que você desenvolva soluções que promovem flexibilidade, facilitam a reutilização de código e reduzem a complexidade dos sistemas. Ao longo da aula, exemplos práticos ilustrarão como aplicar esses conceitos no design de software.

Acoplamento

Hoje, exploraremos conceitos fundamentais de design de software, começando pelo acoplamento. O acoplamento refere-se ao grau de dependência entre classes ou módulos dentro de um sistema, e essa relação afeta diretamente a qualidade do design. Em um sistema com alto acoplamento, as classes são fortemente dependentes umas das outras, o que resulta em um design mais rígido e propenso a erros. Por outro lado, o baixo acoplamento é caracterizado por uma menor dependência entre as classes, permitindo um design mais flexível, fácil de manter e modificar. A distinção entre alto e baixo acoplamento pode ser ilustrada com um exemplo prático.

Consideremos o cenário de um sistema de pedidos em uma loja online, com classes como Pedido, Cliente, Item, Carrinho de Compras e Pagamento. Se a classe Pedido acessar diretamente a classe Cliente para obter informações, e a classe Cliente fizer o mesmo para obter dados sobre os pedidos, estamos diante de um alto acoplamento. Essas duas classes dependem fortemente

uma da outra, de modo que uma mudança em uma impactaria diretamente na outra. Este tipo de acoplamento dificulta a manutenção do código e aumenta a chance de erros.

Agora, para reduzir o acoplamento, podemos modificar o exemplo. Em vez de as classes acessarem diretamente os detalhes umas das outras, podemos usar uma abordagem baseada em interfaces. O Pedido pode, por exemplo, receber um objeto Cliente como parâmetro em um método, como o método “processar”. Dessa forma, a comunicação entre as classes ocorre através da passagem de objetos, sem que uma precise conhecer os detalhes internos da outra. Isso caracteriza um baixo acoplamento, uma vez que a alteração na implementação de uma classe não afetaria a outra, desde que as interfaces permaneçam intactas.

A vantagem do baixo acoplamento é que ele permite que mudanças em um módulo ou classe sejam realizadas sem causar impacto em outros módulos do sistema. Isso facilita a refatoração, a reestruturação e a substituição de partes do sistema sem comprometer seu funcionamento. Por outro lado, no alto acoplamento, qualquer alteração pode exigir mudanças em diversas partes do sistema, aumentando a complexidade e os riscos de introdução de erros.

Um bom design de software é aquele em que os módulos são fracamente acoplados, permitindo uma maior flexibilidade. Essa flexibilidade significa que é possível substituir ou modificar módulos sem comprometer a funcionalidade de outros. A avaliação do grau de acoplamento em um sistema pode ser feita por meio de métricas como o número de conexões entre os módulos e a facilidade com que eles utilizam as interfaces uns dos outros. Quanto menor o número de conexões e maior a facilidade de uso das interfaces, melhor será o design. Além disso, quanto mais intercambiáveis forem os módulos, mais flexível será o sistema.

Coesão

A coesão refere-se ao grau de clareza nas responsabilidades de um módulo ou classe dentro de um sistema de software. Ela indica o quanto os elementos internos de uma classe – como métodos e atributos – estão relacionados e trabalham juntos para realizar uma única responsabilidade. É importante frisar que falamos de “uma única responsabilidade”, o que não deve ser confundido com “uma única função” ou “um único método”. A ideia central é que todos os elementos da classe devem contribuir para a realização dessa responsabilidade de forma unificada, promovendo um design claro e objetivo.

Existem dois tipos principais de coesão: alta e baixa. Na alta coesão, os métodos e atributos de uma classe estão intimamente relacionados e operam sobre os mesmos dados ou dentro do mesmo contexto. Por exemplo, se tivermos uma classe que trata de um conceito específico, como a classe “Círculo”, ela conterá métodos como “calcular área”, “calcular circunferência” e “desenhar círculo”. Todos esses métodos se relacionam diretamente com o conceito de círculo, promovendo a alta coesão, pois estão focados em uma única responsabilidade. Esse é o cenário desejado no design de software.

Por outro lado, a baixa coesão ocorre quando os métodos e atributos de uma classe não estão relacionados ou estão dispersos, resultando em uma confusão de responsabilidades. Um exemplo disso seria uma classe “Utilitário”, que contém métodos para manipular arquivos, formatar strings e realizar cálculos matemáticos. Essas funcionalidades não têm uma relação clara entre si e, portanto, representam baixa coesão. Esse tipo de design torna o sistema mais complexo, difícil de manter e propenso a erros, uma vez que as responsabilidades estão distribuídas de maneira desordenada.

O impacto da baixa coesão pode ser visto em termos de aumento da complexidade do sistema. Quando as responsabilidades estão dispersas, a compreensão do código se torna mais difícil, e a manutenção mais trabalhosa. A reutilização de código também é prejudicada, pois não há uma lógica clara entre as partes do sistema. Um exemplo prático seria uma classe “Controle”, que contém métodos para validar entradas de usuários, processar dados e interagir com um banco de dados. Apesar de parecer coeso à primeira vista, essas responsabilidades são diversas e não estão intimamente ligadas, o que resulta em baixa coesão.

Em contraste, a alta coesão traz benefícios claros para o design de software. Um sistema coeso é mais fácil de entender, de testar e de manter. Como as responsabilidades estão bem definidas dentro de cada classe, é possível realizar modificações localizadas sem impacto em outras partes do sistema. Além disso, a modularidade e flexibilidade do código aumentam significativamente, facilitando a extensão e a adaptação do sistema a novas necessidades. Esse é o cenário ideal para promover um design de software robusto e eficiente.

Decomposição

A decomposição em orientação a objetos é o processo de dividir um sistema complexo em partes menores e mais gerenciáveis. Essas partes podem ser módulos, classes ou funções, que, quando decompostas, tornam o sistema mais organizado e fácil de entender. O princípio da decomposição é especialmente importante à medida que os sistemas se tornam cada vez mais complexos. Dividir o sistema em componentes menores permite que cada parte tenha seu comportamento e dados encapsulados de maneira independente, seguindo os princípios de encapsulamento e coesão. Isso significa que cada módulo deve conter todos os elementos necessários para realizar sua função de maneira clara, sem depender excessivamente de outros módulos, garantindo assim um baixo acoplamento.

Ao decompor um sistema, cada objeto ou módulo tem uma responsabilidade única, o que melhora a modularidade e facilita tanto a reutilização quanto a manutenção do código. Por exemplo, em um sistema de gerenciamento de pedidos, podemos decompor o sistema em módulos como “Pedido”, “Cliente” e “Pagamento”, onde cada um desses módulos têm uma responsabilidade clara e bem definida. Isso facilita a leitura do código, sua compreensão e, principalmente, sua manutenção a longo prazo.

Um ponto importante é não confundir decomposição com abstração. Embora os dois conceitos pareçam similares, eles têm finalidades diferentes. A abstração trata dos aspectos essenciais de um sistema, ignorando detalhes que não são relevantes para o contexto. Ao abstrair, focamos apenas no que

é necessário para o cenário em questão. Por exemplo, se estamos projetando um sistema para gerenciar um estacionamento, as informações essenciais sobre um carro podem incluir a placa e o modelo. Já em um sistema de lava-jato, o tamanho e a cor do carro seriam os aspectos mais importantes. A abstração seleciona apenas o que é relevante para o contexto.

Agora, voltando à decomposição, o objetivo é dividir um sistema em partes menores e mais específicas. Vamos usar um exemplo prático: em um sistema de gerenciamento de animais de estimação, podemos ter uma classe geral chamada “Animal”, que abrange propriedades como nome, idade, e comportamentos como “comer” e “dormir”. A partir dessa classe, podemos decompor o sistema criando subclasses mais específicas, como “Cachorro” e “Gato”. Ambas as subclasses herdam os comportamentos gerais de “Animal”, mas adicionam comportamentos específicos: a classe “Cachorro” pode incluir o método “latir”, enquanto a classe “Gato” incluiria o método “mear”.

Essa abordagem de decomposição ajuda a tornar um sistema complexo mais compreensível e fácil de gerenciar. Ao decompor um sistema em partes menores e mais especializadas, conseguimos não só organizar melhor o código, mas também promover a reutilização de componentes e a flexibilidade do sistema. Cada parte pode ser alterada ou melhorada sem afetar o funcionamento do restante do sistema, desde que os princípios de encapsulamento e coesão sejam respeitados.

Generalização

A generalização, no contexto da orientação a objetos, é o processo de abstrair características comuns de diferentes classes para criar uma classe mais genérica. O objetivo dessa abstração é unificar esses elementos semelhantes em uma estrutura comum, facilitando a reutilização de código e a manutenção. Esse conceito é algo que já observamos na natureza e em outras áreas. Por exemplo, podemos generalizar mamíferos, aves, peixes, anfíbios e répteis dentro de uma classe genérica chamada “Animal”, onde compartilham atributos e comportamentos comuns. Na matemática, um polígono é uma generalização de figuras geométricas como o triângulo e o

quadrilátero, que compartilham a característica de terem lados, mas com diferenças específicas.

A generalização permite que pensemos em um nível mais elevado, abstraindo os detalhes específicos de cada classe ou objeto para nos concentrarmos nas semelhanças essenciais. Isso é fundamental para um design eficiente, pois simplifica a estrutura do sistema ao remover redundâncias. No entanto, é importante não confundir generalização com herança. Embora a herança seja uma forma comum de implementar a generalização, ela não é o único meio. A herança ocorre quando uma classe herda atributos e métodos de outra, criando uma hierarquia de classes. Já a generalização, por si só, é a criação de uma classe genérica a partir de outras, independentemente de usarmos herança para isso ou não.

Um exemplo prático de generalização seria o caso de três classes diferentes: “Tarefa Projeto”, “Tarefa Manutenção” e “Tarefa Suporte”. Ao analisar essas classes, percebemos que elas compartilham certos atributos e métodos. Podemos, então, criar uma classe mais genérica chamada “Tarefa”, que conterá todas essas características comuns. As classes especializadas, como a “Tarefa Projeto”, podem herdar dessa classe genérica ou simplesmente instanciar objetos dela para utilizar seus atributos e métodos. Nesse caso, a herança não é obrigatória, mas a generalização foi aplicada para otimizar o design.

Outro exemplo seria a criação de uma classe generalizada “Veículo”, a partir das classes “Carro” e “Moto”, que compartilham atributos como marca, modelo, cor e ano. Ambas as classes podem instanciar objetos da classe “Veículo” para aproveitar esses atributos, sem que precisem, necessariamente, herdar a classe. Os métodos como “ligar”, “desligar” e “acelerar” também poderiam ser generalizados, enquanto métodos específicos, como “empinar” (para moto) e “acionar freio de mão” (para carro), permaneceriam nas classes especializadas.

Além disso, podemos usar a composição como outra forma de aplicar a generalização. Por exemplo, imagine que já existam as classes “Motor”, “Carroceria” e “Rodas”. A classe “Carro” pode ser composta por essas classes, criando uma nova entidade sem a necessidade de herança. Esse é um

exemplo de como a generalização pode ser implementada de diferentes maneiras, seja por herança ou composição, conforme a necessidade do design.

Com isso, encerramos nossa aula sobre os Princípios de Design, abordando os conceitos de acoplamento, coesão, decomposição e generalização, e como cada um deles contribui para a criação de um sistema de software eficiente e fácil de manter.

Conteúdo Bônus

Para se aprofundar no tema desta aula, sugiro que assista ao vídeo intitulado “Princípios Básicos do Design Gráfico - Guia para iniciantes no design”, que está disponível no canal Leo Becker no YouTube.

Referência Bibliográfica

ASCENCIO, A. F. G.; CAMPOS, E. A. V. de. **Fundamentos da programação de computadores**. 3.ed. Pearson: 2012.

BRAGA, P. H. **Teste de software**. Pearson: 2016.

GALLOTTI, G. M. A. (Org.). **Arquitetura de software**. Pearson: 2017.

GALLOTTI, G. M. A. **Qualidade de software**. Pearson: 2015.

MEDEIROS, E. **Desenvolvendo software com UML 2.0 definitivo**. Pearson: 2004.

MORAIS, I. S. de. (Org.). **Engenharia de software**. Pearson: 2017.

PFLEEGER, S. L. **Engenharia de software: teoria e prática**. 2.ed. Pearson: 2003.

SOMMERVILLE, I. **Engenharia de software**. 10.ed. Pearson: 2019.